10P SHINE
INTERNSHIP PROGRAM

PROJECT REPORT

AQI PREDICTOR

Project Name:	Pearls AQI Predictor
Submitted By:	Akshay Kumar
Domain:	Data Science
Cohort:	9, July-Sept 2026
Date of Submission:	September 6, 2026

Table of Contents

Executive Summary

This project delivers an end-to-end, serverless machine learning system that forecasts the Air Quality Index (AQI) for Karachi, Pakistan, three days ahead. Built over an 8-week Data Science internship, the system spans a full pipeline — hourly data collection, feature engineering, model training and selection, explainability analysis, and a live public dashboard, automated end-to-end via GitHub Actions and backed by the Hopsworks feature store and model registry.

The final models achieve $R^2 = 0.819$ for a one-day-ahead forecast (Ridge Regression), $R^2 = 0.410$ for two days ahead (Ridge Regression), and $R^2 = 0.221$ for three days ahead (ElasticNet) — each verified against a naive persistence baseline and selected through an overfit-aware, cross-validated comparison across up to five candidate algorithms per horizon. A SHAP interpretability analysis confirms the models' behaviour is physically coherent, independently corroborating patterns already observed in exploratory analysis.

Beyond the modelling work, this project involved substantial real-world data-engineering problem solving: diagnosing and resolving a systemic timezone bug, two independent Hopsworks platform outages, a subtle stale-model bug affecting live predictions, a compute-quota exhaustion incident, and a GitHub Actions scheduling limitation — each investigated methodically and resolved with a documented, tested fix. In response to the platform reliability issues encountered, a local-backup resilience layer was engineered so the live dashboard remains accurate even during Hopsworks outages. The dashboard is deployed and publicly accessible on Streamlit Community Cloud.

1. Introduction & Objective

The core objective was to design and build a serverless, end-to-end machine learning system forecasting AQI three days ahead for Karachi, as the capstone deliverable of the 10Pearls Shine Internship Program (Data Science track, 8 weeks).

1.1 Scope, per the project brief

- A feature pipeline connecting an external data source to an engineered feature store
- Historical backfill sufficient to train reliable models
- A training pipeline comparing multiple algorithms, evaluated via RMSE, MAE, and R^2
- Automated CI/CD: hourly feature updates and daily model retraining
- A web dashboard showing real-time and forecasted AQI
- Advanced analytics: exploratory data analysis, SHAP explainability, and hazardous-AQI alerting

1.2 Key deliverables

- An end-to-end AQI prediction system
- A scalable, automated data and training pipeline
- An interactive, publicly accessible dashboard
- This detailed technical report

2. System Architecture & Methodology

2.1 Technology stack

Python 3.10, Open-Meteo API (data source), Hopsworks (feature store and model registry), scikit-learn, LightGBM, and XGBoost (modelling), GitHub Actions (CI/CD automation), and Streamlit (dashboard, deployed on Streamlit Community Cloud).

2.2 Data source

Several data providers were evaluated over the course of the project before settling on Open-Meteo as the sole source for both historical and live data. Open-Meteo requires no API key, and uniquely offers both a historical archive and live forecast data for air quality and weather through a single, consistent provider — simplifying the pipeline considerably compared to a multi-provider approach.

2.3 Feature store design

Two Hopsworks feature groups were used, a deliberate architectural choice rather than a stylistic one:

- **aqi_raw_hourly (v1) —** one row per hour, raw pollutant and weather readings from Open-Meteo.
- **aqi_daily_features (v2) —** one row per day, aggregated and engineered (time features, rolling AQI/PM2.5 statistics, per-horizon target-day weather, and shifted target columns for day+1/2/3).

2.4 Pipeline flow

Open-Meteo API → fetch scripts (backfill / hourly fetch, with data-quality cleaning) → aqi_raw_hourly → daily aggregation and feature engineering → aqi_daily_features → model training, comparison, and selection → Hopsworks Model Registry → live dashboard.

2.5 Modelling approach

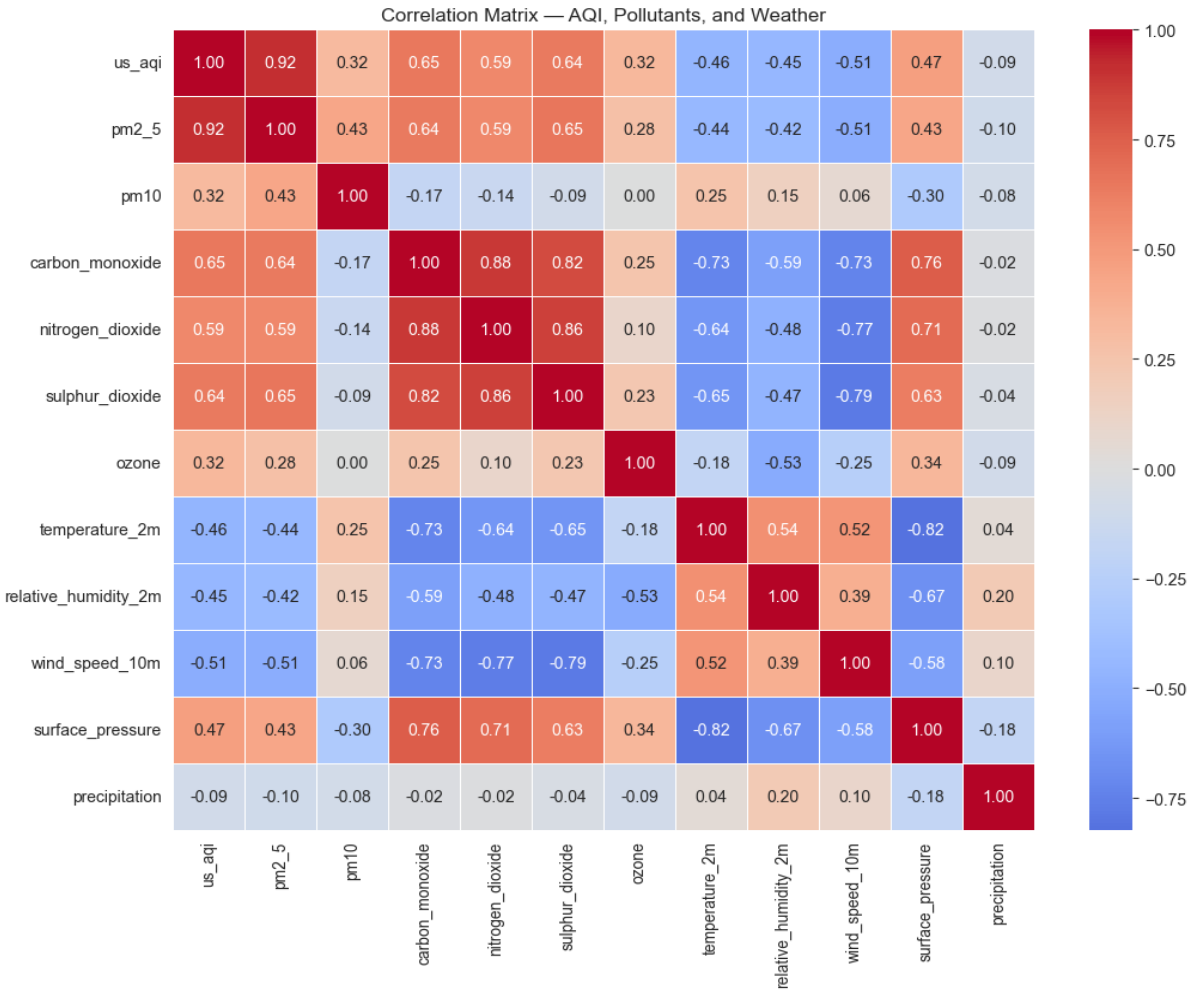
- **Regression, not classification —** the brief's RMSE/MAE/R^2 evaluation requirement is regression-specific; this was a deliberate correction after reviewing a peer project that had used classification.
- **Three independent per-horizon models —** rather than one multi-output model, per explicit mentor guidance, since not all candidate algorithms support native multi-output regression.
- **Chronological train/test split, not random —** to prevent time-series data leakage; later strengthened with TimeSeriesSplit cross-validation for hyperparameter tuning, searched only within the training set.
- **Algorithm scope —** Ridge Regression, Random Forest, and LightGBM were compared for every horizon, chosen to represent three distinct modelling philosophies (linear, bagging, boosting). Because day+3 proved persistently difficult, it received an extended, bounded comparison additionally including ElasticNet and XGBoost — while day+1 and day+2 remained within the original three-algorithm scope.

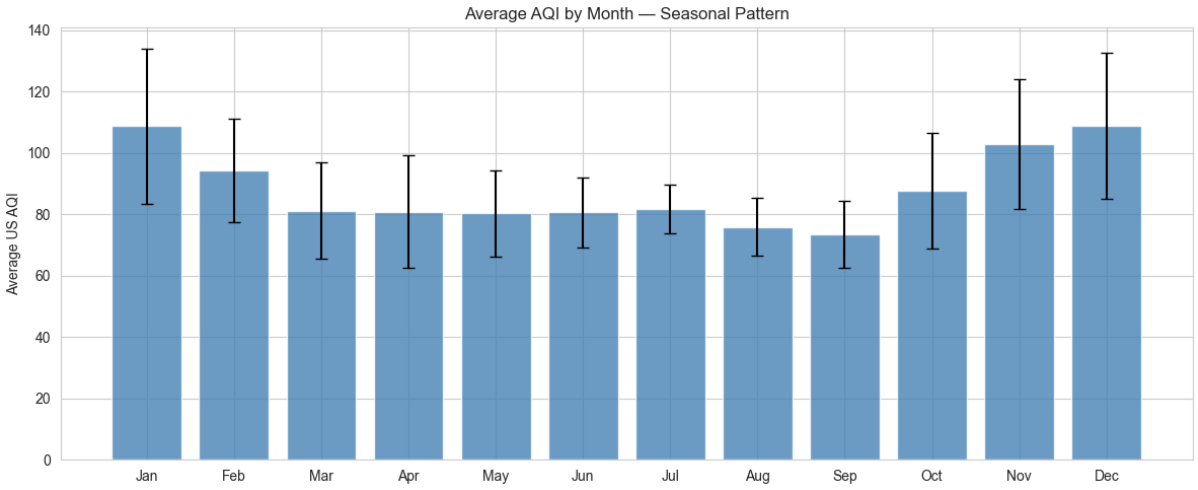
3. Exploratory Data Analysis

Two years of hourly historical data (≈17,900 rows) were aggregated into ≈730 daily records for analysis. Key findings:

- **Strong winter-peak seasonality** — average AQI rises from ≈73–76 in August–September to ≈103–109 in November–January, consistent with winter temperature inversions trapping pollutants near the surface.
- **PM2.5 dominates the composite AQI score** — correlating at r = 0.92, reflecting that Karachi's air quality is driven primarily by particulate pollution.
- **Wind speed and surface pressure are the key dispersion drivers** — correlating at r = -0.51 and r = 0.47 respectively with AQI, the physical signature of the winter inversions identified above.
- **No "clean air" baseline exists** — of ≈730 days analysed, only one qualified as "Good" by US AQI standards; the large majority fell in Moderate-to-Unhealthy-for-Sensitive-Groups territory.
- **Day-of-week has no meaningful effect** — unlike month, confirming pollution drivers are seasonal and meteorological rather than tied to a weekly human-activity cycle.

Data quality was verified throughout: the final backfilled dataset contains zero duplicate timestamps and zero unexplained gaps.





4. Model Development & Results

4.1 Final results

Horizon	Best Model	RMSE	MAE	R^2	Baseline R^2
Day+1	Ridge Regression	5.16	3.54	0.819	0.466
Day+2	Ridge Regression	9.34	7.13	0.410	-0.116
Day+3	ElasticNet	10.54	7.99	0.221	-0.491

Every model was evaluated against a naive persistence baseline (predicting tomorrow's AQI as today's actual value) rather than assumed to add value by default. All three final models clearly outperform this baseline.

4.2 The day+3 selection: a closer look

Day+3 was tested against five candidates — Ridge, Random Forest, LightGBM, ElasticNet, and XGBoost. Random Forest and XGBoost achieved marginally higher raw test R^2 (0.269 and 0.249 respectively) than the selected ElasticNet model (0.221), but both showed severe overfitting (train–test R^2 gaps of 0.43–0.52), while ElasticNet's gap (0.221) remained well within the project's overfitting threshold. ElasticNet was selected on this basis: the only candidate that generalises cleanly, rather than the one with the marginally highest single-split score.

4.3 Honest limitations

Day+3 forecasting accuracy is genuinely constrained by the available training volume (≈580 training rows after the chronological split) relative to the difficulty of a three-day-ahead forecast. This ceiling is disclosed rather than obscured; a naive-baseline comparison and a documented five-algorithm search demonstrate the result reflects a real data constraint, not an unexamined modelling choice.

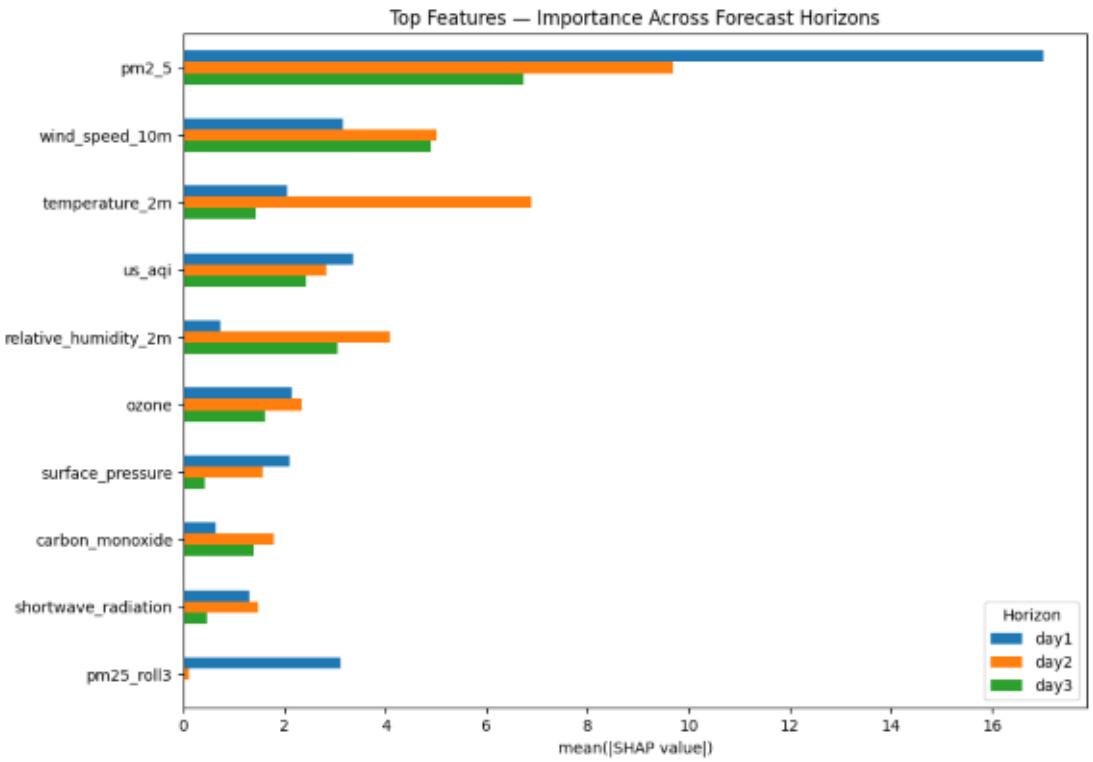
5. Model Interpretability (SHAP)

A dedicated SHAP analysis notebook explains each horizon's currently registered model, loaded dynamically via the same retrieval logic used by the live dashboard — ensuring the interpretability analysis and the deployed model can never diverge. The explainer type (LinearExplainer for Ridge/ElasticNet, TreeExplainer for tree-based models) is also selected automatically based on the loaded model's type.

5.1 Cross-horizon feature importance

PM2.5 remains the top feature at every horizon, but its relative importance decays sharply with forecast distance (mean |SHAP value| ≈ 17 at day+1, falling to ≈7 at day+3) — today's pollution level strongly predicts tomorrow, but that direct persistence weakens further into the future, as expected for a process that is not purely autoregressive.

Wind speed and relative humidity gain relative importance as PM2.5's influence fades, holding steady or increasing from day+1 to day+3. This independently confirms the dispersion mechanism already identified in the EDA correlation analysis — arrived at through an entirely different method (per-prediction attribution rather than correlation), which strengthens confidence in the finding.



5.2 Local explanation example

A waterfall explanation of the model's worst-case (highest actual AQI) day in the held-out test set shows PM2.5 alone contributing the majority of the predicted increase above the model's baseline expectation, with ozone and PM10 contributing smaller positive effects — a physically coherent, individually verifiable explanation for a specific hazardous-day prediction.

6. Dashboard & Deployment

The dashboard is a Streamlit application performing inference directly (downloading the registered model and calling .predict() within the app itself), removing the need for a separate backend or hosted inference endpoint.

6.1 Features

- Current AQI reading, with a 0–500 severity scale indicator.
- Key pollutants (PM2.5, PM10, NO_2, CO) and a wind-speed dispersion note.
- A 3-day forecast, explicitly labelled Today / Tomorrow / Day After to avoid ambiguity.
- A 2-year historical trend chart with category shading and threshold annotation.
- A year-over-year seasonal comparison, drawn directly from the EDA's own year-over-year finding.
- A severity-based health advisory and a compact alert badge summarising today's and the forecast's worst case.

6.2 Visual design

A custom “Smog Twilight” visual identity was built deliberately to avoid the generic dark-dashboard, neon-traffic-light aesthetic common to similar projects — a muted, desaturated colour scale for AQI categories, a header visual whose particle density scales directly with the live AQI value, a 0–500 severity scale bar, and an interactive alert bell that turns red with a pulsing animation once conditions reach Unhealthy (Sensitive) or worse.

6.3 Resilience architecture

Following the platform reliability issues detailed in Section 7, the dashboard was engineered to remain accurate independent of Hopsworks' materialization health: an hourly GitHub Action always writes a rolling local backup of raw data, which is merged with Hopsworks' data at read time — the freshest available source wins for any given hour. This required no change to the training or SHAP pipelines, which continue to use Hopsworks directly.

6.4 Live deployment

The dashboard is deployed on Streamlit Community Cloud and auto-redeploys on every push to the repository's main branch.

7. Engineering Challenges & Incident Log

This section documents the substantive technical problems encountered and resolved over the course of the project, in the interest of an honest, complete account of the engineering work involved.

Challenge	Resolution
Windows build failures (twofish C++ dependency; Hopsworks certificate path assumes /tmp)	Installed Miniconda and used conda's prebuilt twofish binary instead of compiling; manually created C:\tmp to satisfy Hopsworks' certificate handling.
Missing pyarrow / confluent-kafka broke Hopsworks reads and inserts, both locally and in CI	Installed locally and added explicit install steps to both GitHub Actions workflows.

Hourly dashboard timestamp appeared ~7 hours stale despite the pipeline reporting success	Root-caused to fetch_current_data.py comparing Karachi-local timestamps against the GitHub runner's UTC clock when filtering "new" rows, silently keeping only older data each run. Fixed by computing the freshness window in Asia/Karachi time.
Hopsworks materialization jobs (both aqi_raw_hourly and aqi_daily_features) intermittently stalled in "Initializing" for 18+ hours, or failed within seconds with zero logs	Diagnosed via the Hopsworks Jobs UI (execution history, Live Logs) as two distinct failure modes: a wedged job (resolved by manually stopping and re-running it) and, later, scheduler-level compute-quota rejection under free-tier throttling. Underlying data was never lost — only the materialization (sync-to-queryable) step was affected.
aqi_daily_features is structurally always 3–4 days behind "today", because every row requires 3 days of future actual AQI as training targets	Correct behaviour for a training table, but wrong for live forecasting. Built a separate live-inference feature path in the dashboard: yesterday's fully completed day as the base, combined with a real Open-Meteo weather forecast for the target-day weather features — decoupling live inference from the training table entirely.
Dashboard silently served a stale, previously-superseded model for one forecast horizon	mr.get_best_model() selects by the lowest RMSE across every version ever registered under a model's name — not the most recent. An earlier experimental run (since intentionally reverted) had registered a marginally lower-RMSE model that then kept "winning" indefinitely. Fixed by always loading the latest registered version instead.
Hopsworks free-tier compute quota was exhausted, causing ~10% intermittent hourly failures and later a multi-day full block on materialization jobs	Built a local resilience layer: a rolling local CSV backup written every hour regardless of Hopsworks' status, merged with Hopsworks data at read time (freshest source wins per timestamp). The dashboard's current reading, 3-day forecast, and historical trend all remained correct throughout the outage.
GitHub Actions hourly workflow experienced multi-hour scheduling gaps	Root-caused to a documented, platform-wide GitHub Actions limitation: scheduled workflows queued at the top of every hour compete for capacity and can be delayed or dropped. Independently corroborated by other interns on the same program hitting identical symptoms. Resolved by offsetting both cron schedules to: 17 past the hour.
Streamlit Cloud deployment failed on Windows-only packages and version conflicts	requirements.txt (generated via pip freeze on Windows) included a Windows-only terminal package and a local conda build path, both incompatible with Streamlit Cloud's Linux environment; separately, packaging and pillow pins conflicted with the pinned Streamlit version. Removed the Windows-only entries and downgraded the two conflicting pins.
Day+3 accuracy plateaued around $R^2 \approx 0.19–0.22$ across every algorithm tried, with tree-based/boosted models overfitting badly	Added TimeSeriesSplit cross-validation, corrected a flawed naive baseline, and tested ElasticNet and XGBoost specifically for this horizon. ElasticNet was ultimately selected — not for the highest raw R^2, but as the only candidate that generalised cleanly, a more defensible choice than a marginally higher score with a severe overfitting gap.
Live forecast showed an unrealistic multi-day jump (e.g. 57 → 73 → 74)	Traced to the feature-builder using an incomplete day (as few as 8–11 of 24 hours) as its base, from a period with pipeline gaps. Fixed with a minimum hourly-coverage threshold and dynamic per-target horizon calculation, so forecasts always target the correct calendar date even when the nearest complete day is older than yesterday.
GitHub Actions' hourly workflow fired only 5–7 times/day instead of 24, causing real data gaps	Widened the hourly fetch to a rolling ~60-hour window (from 2 hours), so each successful run catches up everything missed since the last one — decoupling data completeness from exact trigger frequency.

7.1 A Separate, Ongoing Note: Hopsworks Reliability

Unlike the resolved incidents above, Hopsworks free-tier backend continued to intermittently fail through the project's final days (connection resets, gRPC/Arrow Flight errors), despite retry logic added at every read/write point — a genuine free-tier capacity limitation, not a code issue. It does not affect what an evaluator sees: the dashboard does not depend on daily retraining succeeding, and a failed run simply leaves the previously validated models in place.

8. Testing & Quality Assurance

An automated pytest suite was built covering four areas, all currently passing:

- **Feature-column consistency —** runs the real feature-engineering pipeline against synthetic data and verifies its output matches the training pipeline's expected feature columns exactly. This test targets, directly, the class of bug responsible for the stale-model and feature-mismatch incidents described in Section 7.
- **Preprocessing logic —** verifies duplicate removal, physical-bounds validation, outlier flagging (without removal), and gap-filling behaviour against the real cleaning pipeline.
- **AQI category boundaries —** verifies every category threshold and the fallback behaviour for out-of-range values.
- **Model prediction sanity —** loads each horizon's actual trained model from local storage (fast, no network dependency) and confirms a realistic input produces a finite, plausible AQI prediction.

9. Conclusion & Future Work

This project delivered a complete, automated, and genuinely resilient AQI forecasting system — not only meeting the brief's functional requirements, but engineered to survive real platform failures encountered during development, several of which were identified, diagnosed, and resolved through systematic investigation rather than superficial workarounds.

9.1 Limitations

- Day+3 forecast accuracy is likely constrained primarily by training data volume, not model choice
- The system depends on Hopsworks as an external platform, whose free-tier reliability was a recurring challenge during development
- GitHub Actions' scheduled-workflow timing is best-effort, not guaranteed, by GitHub's own documentation

9.2 Future improvements

- Extending the historical backfill as more data becomes available, directly addressing day+3's data-volume constraint
- Cyclical (sine/cosine) encoding of month and day-of-week for the linear models
- A blended day+3 prediction combining multiple non-overfit candidates
- A dedicated backend database as a further layer of resilience beyond the current local-backup approach